

01_estructura-general-del-proyecto

[#linux](#)[#bash](#)[#python](#)[#fastapi](#)[#docker](#)[#backend](#)[#frontend](#)[#PostgreSQL](#)[#SIEM](#)

1 Objetivo de la nota

Esta nota analiza la estructura general del proyecto `siem-lab`.

El objetivo es entender qué carpetas y archivos forman parte del laboratorio, qué responsabilidad tiene cada bloque y cómo se relacionan entre sí.

Esta nota sirve como mapa técnico del proyecto antes de entrar en archivos concretos del entorno, dependencias, variables de entorno o ejecución.

2 Ubicación del proyecto

El proyecto se encuentra en:

```
~/siem-lab
```

La terminal muestra que la ruta de trabajo es:

```
/home/endurance/siem-lab
```

Para entrar al proyecto se utiliza:

```
cd ~/siem-lab
```

Este directorio es la raíz del laboratorio SIEM MVP.

3 Comandos utilizados para revisar la estructura

Para revisar la estructura se ejecutaron varios comandos:

```
cd ~/siem-lab

tree -a -L 4

cat .env.example

find backend -maxdepth 4 -type f | sort

find frontend -maxdepth 3 -type f | sort

find docker -maxdepth 3 -type f | sort

ls -la
ls -la backend
ls -la backend/app
ls -la frontend
ls -la docker
```

El comando `tree` no estaba instalado, por lo que no pudo mostrar el árbol visual.

El sistema indicó:

```
Command 'tree' not found
```

Aun así, la estructura se pudo revisar correctamente usando:

```
find
ls -la
```

Estos comandos sí mostraron los archivos, carpetas y permisos principales del proyecto.

4 Estructura general detectada

En la raíz del proyecto aparecen estos elementos:

```
siem-lab/
├─ backend/
├─ configs/
├─ data/
├─ docker/
├─ .env
├─ .env.example
├─ frontend/
├─ .git/
├─ .github/
├─ .gitignore
└─ README.md
```

La estructura separa claramente:

```
backend
  ↓
lógica de API, base de datos, modelos, schemas, migraciones y tests

frontend
  ↓
interfaz web HTML, CSS y JavaScript

docker
  ↓
ejecución contenerizada del laboratorio

.env.example
  ↓
plantilla de configuración

README.md
  ↓
documentación principal

.gitignore
  ↓
exclusión de archivos no entregables
```

5 Lectura general de la raíz

La salida de `ls -la` muestra:

```
backend/
configs/
data/
docker/
.env
.env.example
frontend/
.git/
.github/
```

```
.gitignore
README.md
```

Esto permite diferenciar tres tipos de elementos:

1. Código fuente del proyecto.
2. Configuración y documentación.
3. Archivos locales o generados que no deben entregarse.

6 Elementos principales del proyecto

backend/

Contiene toda la parte de servidor.

Es el núcleo de la lógica del laboratorio:

```
API FastAPI
modelos SQLAlchemy
schemas Pydantic
migraciones Alembic
tests pytest
Dockerfile
requirements.txt
```

frontend/

Contiene la interfaz web.

Está construida con:

```
HTML
CSS
JavaScript puro
```

No usa frameworks como React, Vue o Angular.

docker/

Contiene la configuración de Docker Compose.

Su objetivo es levantar los servicios del laboratorio de forma reproducible.

.env.example

Es la plantilla de variables de entorno.

Este archivo sí debe entregarse porque permite reconstruir la configuración sin exponer valores reales.

.env

Es un archivo real de entorno.

No debe entregarse porque puede contener configuración local o sensible.

.git/

Contiene el repositorio Git interno.

No debe incluirse en el `.zip` de entrega.

.github/

Contiene configuración asociada a GitHub.

Puede entregarse si forma parte del proyecto.

.gitignore

Define qué archivos debe ignorar Git.

Es importante porque documenta qué elementos no forman parte del código fuente.

README.md

Es la documentación principal del proyecto.

En la salida aparece actualizado recientemente:

```
README.md 11087 May 26 15:31
```

Esto indica que forma parte de la entrega final.

7 Estructura del backend

El directorio `backend/` contiene:

```
backend/  
├── alembic/  
├── alembic.ini  
├── app/  
├── Dockerfile  
├── .pytest_cache/  
├── pytest.ini  
├── requirements.txt  
├── tests/  
└── venv/
```

Esta estructura muestra que el backend está preparado para:

```
desarrollo local  
ejecución con Docker  
migraciones de base de datos  
pruebas automatizadas
```

8 Elementos principales del backend

backend/app/

Contiene el código principal de la aplicación.

Es la parte más importante del backend.

backend/alembic/

Contiene las migraciones de base de datos.

Permite versionar la evolución de las tablas.

backend/alembic.ini

Archivo de configuración de Alembic.

backend/Dockerfile

Define cómo construir la imagen Docker del backend.

backend/requirements.txt

Lista las dependencias Python necesarias.

backend/pytest.ini

Configura pytest.

backend/tests/

Contiene tests automatizados.

backend/venv/

Entorno virtual local.

No forma parte del código fuente que debe entregarse.

backend/.pytest_cache/

Caché generada por pytest.

No debe entregarse.

9 Estructura de backend/app/

La carpeta principal de la aplicación contiene:

```
backend/app/
├── api/
├── db/
├── __init__.py
├── main.py
├── models/
├── __pycache__/
└── schemas/
```

Esta separación es típica en un backend organizado.

Cada subcarpeta tiene una responsabilidad concreta:

```
api/
  ↓
rutas HTTP

db/
  ↓
conexión y sesiones de base de datos

models/
  ↓
modelos ORM

schemas/
  ↓
validaciones y respuestas

main.py
```

↓
punto de entrada de FastAPI

10 Archivo `main.py`

El archivo:

```
backend/app/main.py
```

es el punto de entrada principal del backend.

Su función es crear la aplicación FastAPI y registrar los routers.

Relación:

```
main.py
  ↓
crea app = FastAPI(...)
  ↓
configura CORS
  ↓
incluye routers
  ↓
expone endpoints
```

11 Carpeta `api/`

La carpeta `backend/app/api/routes/` contiene los endpoints de la API:

```
alerts.py
events.py
health.py
info.py
ingest.py
metrics.py
rules.py
```

Relación funcional:

```
health.py
  ↓
comprueba salud de backend y base de datos

info.py
  ↓
devuelve información de versión y build

events.py
  ↓
consulta eventos

ingest.py
  ↓
recibe eventos y genera alertas

rules.py
  ↓
crea y lista reglas

alerts.py
  ↓
consulta y actualiza alertas
```

```
metrics.py
  ↓
devuelve métricas agregadas
```

Estos archivos forman la capa API del laboratorio.

1 2 Carpeta db/

La carpeta:

```
backend/app/db/
```

contiene:

```
base.py
database.py
session.py
```

Función:

```
base.py
  ↓
define la clase Base para SQLAlchemy

database.py
  ↓
configura conexión con PostgreSQL

session.py
  ↓
proporciona sesiones de base de datos a FastAPI
```

Este bloque permite que los endpoints trabajen con PostgreSQL.

1 3 Carpeta models/

La carpeta:

```
backend/app/models/
```

contiene:

```
alert.py
event.py
rule.py
```

Estos archivos definen las tablas principales del sistema:

```
event.py
  ↓
tabla events

rule.py
  ↓
tabla rules

alert.py
  ↓
tabla alerts
```

Relación del SIEM:

```
Event
  ↓
se evalúa contra

Rule
  ↓
si coincide genera

Alert
```

14 Carpeta `schemas/`

La carpeta:

```
backend/app/schemas/
```

contiene:

```
alert.py
event.py
ingest.py
rule.py
```

Los schemas definen cómo entra y sale la información de la API.

Relación típica:

```
JSON recibido
  ↓
schema Pydantic
  ↓
modelo SQLAlchemy
  ↓
base de datos
  ↓
schema de salida
  ↓
JSON devuelto
```

15 Carpeta `alembic/`

La carpeta `backend/alembic/` contiene migraciones.

En la salida aparecen migraciones como:

```
create_events_table
add_meta_to_events
add_rules_and_alerts
add_group_key_to_alerts
add_throttle_to_rules
add_threshold_to_rules
add_status_and_updated_at_to_alerts
fix_group_key_default
```

Esto permite ver la evolución del proyecto:

1. Se crea la base inicial.
2. Se añade tabla de eventos.
3. Se añade meta JSON a eventos.

4. Se crean reglas y alertas.
5. Se añade group_key.
6. Se añade throttle.
7. Se añade threshold.
8. Se añade ciclo de vida de alertas.

Esta evolución demuestra que el proyecto fue creciendo por iteraciones.

16 Carpeta tests/

El proyecto contiene tests automatizados:

```
backend/tests/test_alerts_ui.py
backend/tests/test_health.py
```

Esto indica que hay validación automatizada, al menos para:

```
healthcheck
endpoints de alertas UI
```

También existe:

```
backend/pytest.ini
```

Por tanto, el proyecto tiene soporte para pytest.

Este bloque se analizará después en el apartado de pruebas y validación.

17 Carpeta frontend/

La carpeta frontend/ contiene:

```
frontend/
├─ alert.html
├─ assets/
└─ index.html
```

Archivos principales:

```
index.html
  ↓
página principal de alertas

alert.html
  ↓
detalle de una alerta

assets/
  ↓
JavaScript y CSS
```

El frontend se mantiene separado del backend.

Esto facilita entender el proyecto en dos capas:

```
backend
  ↓
sirve API

frontend
```

↓
consume API

18 Carpeta frontend/assets/

Dentro de frontend/assets/ aparecen:

```
app.js
alerts.js
alert_detail.js
styles.css
```

Función:

```
app.js
  ↓
funciones comunes y apiFetch()

alerts.js
  ↓
listado de alertas, filtros y paginación

alert_detail.js
  ↓
detalle de alerta y cambio de estado

styles.css
  ↓
diseño visual
```

Este frontend no necesita compilación.

Puede servirse directamente con:

```
python3 -m http.server 5173
```

desde la carpeta frontend.

19 Carpeta docker/

La carpeta docker/ contiene:

```
docker/
├── compose.yml
└── .env
```

El archivo importante para reproducir el entorno es:

```
docker/compose.yml
```

El archivo:

```
docker/.env
```

es configuración real local y no debería entregarse.

La estructura Docker permite levantar servicios como:

```
API FastAPI
PostgreSQL
Adminer
```

Relación:

```
docker/compose.yml
↓
lee variables
↓
construye backend con Dockerfile
↓
levanta PostgreSQL
↓
expone API y Adminer
```

20 Archivos de entorno

Existen tres archivos relacionados con variables:

```
.env
.env.example
docker/.env
```

El archivo entregable es:

```
.env.example
```

Contenido mostrado:

```
POSTGRES_DB=siem
POSTGRES_USER=siem
POSTGRES_PASSWORD=change_me
DATABASE_URL=postgresql+psycopg://siem:change_me@db:5432/siem

API_PORT=8000
ADMINER_PORT=8080

APP_VERSION=0.1.0
GIT_SHA=unknown
BUILD_TIME=unknown
```

Interpretación:

```
POSTGRES_DB
↓
nombre de base de datos

POSTGRES_USER
↓
usuario de PostgreSQL

POSTGRES_PASSWORD
↓
contraseña de ejemplo

DATABASE_URL
↓
cadena de conexión SQLAlchemy

API_PORT
↓
puerto de FastAPI

ADMINER_PORT
↓
```

```
puerto de Adminer

APP_VERSION
  ↓
versión de aplicación

GIT_SHA
  ↓
referencia del commit

BUILD_TIME
  ↓
momento de build
```

2.1 Carpetas generadas automáticamente

En la salida aparecen carpetas generadas:

```
__pycache__/
.pytest_cache/
venv/
```

También aparecen archivos:

```
*.pyc
```

Estos elementos no forman parte del código fuente.

Se generan por:

```
Python
pytest
entorno virtual local
```

No deben entregarse en el ZIP.

2.2 Diferencia entre estructura de trabajo y estructura de entrega

La estructura de trabajo contiene todo lo que se usa durante el desarrollo.

La estructura de entrega debe estar limpia.

En desarrollo pueden existir:

```
.env
docker/.env
.git/
backend/venv/
__pycache__/
.pytest_cache/
*.pyc
```

En la entrega deben excluirse:

```
.env
docker/.env
.git/
backend/venv/
__pycache__/
.pytest_cache/
*.pyc
```

En la entrega deben incluirse:

```
backend/app/
backend/alembic/
backend/alembic.ini
backend/Dockerfile
backend/requirements.txt
backend/pytest.ini
backend/tests/
frontend/
docker/compose.yml
.env.example
.gitignore
README.md
```

Esta distinción es importante porque permite entregar un proyecto reproducible sin archivos locales innecesarios.

2 3 Relación funcional entre carpetas

La estructura completa puede entenderse así:

```
siem-lab/
├── backend/
│   ├── app/
│   │   ├── api/
│   │   ├── db/
│   │   ├── models/
│   │   └── schemas/
│   ├── alembic/
│   └── tests/
├── frontend/
│   ├── index.html
│   ├── alert.html
│   └── assets/
└── docker/
    └── compose.yml
```

Relación funcional:

```
docker/
  ↓
levanta servicios

backend/
  ↓
expone API y conecta con PostgreSQL

frontend/
  ↓
consume API y muestra alertas

tests/
  ↓
valida partes del backend

alembic/
  ↓
gestiona evolución de base de datos
```

2 4 Flujo técnico desde la estructura

La estructura del proyecto refleja el flujo del SIEM:

```
frontend/
  ↓
usuario consulta alertas

backend/app/api/routes/
  ↓
endpoints reciben peticiones

backend/app/schemas/
  ↓
validan entrada y salida

backend/app/models/
  ↓
representan tablas

backend/app/db/
  ↓
gestiona conexión a PostgreSQL

backend/alembic/
  ↓
versiona la estructura de base de datos

docker/
  ↓
orquesta la ejecución
```

Flujo de ejemplo:

```
Usuario abre index.html
  ↓
frontend/assets/alerts.js
  ↓
GET /alerts
  ↓
backend/app/api/routes/alerts.py
  ↓
modelo Alert
  ↓
PostgreSQL
  ↓
respuesta JSON
  ↓
tabla HTML
```

2.5 Estructura y mantenibilidad

La estructura es adecuada para un MVP porque separa responsabilidades:

```
API
  ↓
routes/

Datos
  ↓
models/

Validación
  ↓
schemas/

Base de datos
```

```
↓
db/ + alembic/

Interfaz
↓
frontend/

Ejecución
↓
docker/

Pruebas
↓
tests/
```

Esto evita tener toda la lógica mezclada en un único archivo.

Aunque el proyecto sea pequeño, ya mantiene una organización parecida a la de una aplicación real.

2.6 Punto importante sobre `configs/` y `data/`

En la raíz aparecen:

```
configs/
data/
```

En la revisión actual no se han mostrado archivos dentro de esas carpetas.

Pueden interpretarse como carpetas preparadas para evolución futura.

Posibles usos:

```
configs/
↓
configuraciones adicionales del laboratorio

data/
↓
datos de prueba, datasets, logs o entradas simuladas
```

Aunque estén vacías, pueden formar parte de la estructura prevista.

2.7 Punto importante sobre `.github/`

La carpeta:

```
.github/
```

aparece en la raíz.

Puede contener configuración relacionada con GitHub.

Por ejemplo:

```
workflows
plantillas
acciones automáticas
```

No se ha mostrado su contenido, pero al estar fuera de `.git/`, puede formar parte del código del proyecto.

2.8 Resumen técnico

La estructura general del proyecto `siem-lab` está organizada en bloques claros:

```
backend/  
  ↓  
lógica de servidor, API, modelos, schemas, migraciones y tests  
  
frontend/  
  ↓  
interfaz web de consulta y gestión de alertas  
  
docker/  
  ↓  
orquestación de servicios  
  
.env.example  
  ↓  
plantilla de configuración  
  
README.md  
  ↓  
documentación principal
```

El proyecto diferencia correctamente entre:

```
código fuente  
configuración reproducible  
archivos locales  
archivos generados automáticamente
```

Esta estructura permite desarrollar, ejecutar, probar y entregar el laboratorio de forma ordenada.